

UNIVERSITY OF ENGINEERING & TECHNOLOGY
PESHAWAR



Department of Electrical Engineering

CEP REPORT

Complex Engineering Problem (CEP)

Smart Traffic Management System v3.0

C++ Programming | CS-1xx Lab

Course	EE-170L-Computer Programming	
Submitted To	Eng. Rizwan	
Department	Electrical Engineering	
Section	B	
Prepared by	Safi Ullah	0686
	M.Owais	0708
	Jawad Ahmad	0674
	Hikmat Ullah Jan	0662
	Abdul Wahab	0708
	Asad Ullah Z	0686

Abstract

The Smart Traffic Management System (STMS) v3.0 is a comprehensive console-based simulation developed in C++ as a Complex Engineering Problem (CEP) for the Programming Fundamentals course at UET Peshawar. The system models a realistic urban traffic environment consisting of 8 intersections, 60 vehicles, 12 traffic officers, 6 fuel stations, 5 public bus routes, and 4 road construction zones.

All simulation data is randomly generated at runtime using structured random seeding, eliminating the need for user data entry. The program demonstrates mastery of core C++ programming concepts including conditional statements, switch-case structures, one-dimensional (1D), two-dimensional (2D), and three-dimensional (3D) arrays, user-defined functions, built-in library functions, and C++ structures (structs).

The system provides 18 operational modules covering weather monitoring, vehicle tracking, speed violation detection, traffic signal management, accident reporting, toll collection, emergency dispatch, police patrol scheduling, legal compliance checks, fuel station status, and a comprehensive final city traffic health report. The project successfully integrates real-world traffic management logic with fundamental programming constructs taught in the course curriculum.

1. Introduction

Urban traffic management is one of the most critical challenges faced by modern cities. Inefficient traffic flow, accidents, rule violations, and resource mismanagement lead to significant economic losses, safety hazards, and environmental degradation. Intelligent traffic systems that can monitor, analyze, and respond to dynamic road conditions are therefore of great practical importance.

This project implements a console-based Smart Traffic Management System (STMS) in standard C++ as a Complex Engineering Problem (CEP). Rather than being a simple exercise, the CEP is designed to push students to integrate multiple programming concepts simultaneously to solve a non-trivial, real-world-inspired problem. The system simulates an entire city traffic network for a single hour, generating randomized but internally consistent data across all subsystems.

1.1 Motivation

The motivation behind this project stems from the need to apply classroom programming theory to a problem of realistic complexity. Traffic management involves simultaneous tracking of multiple entities (vehicles, officers, signals, intersections), application of rules and penalties, data aggregation across 2D and 3D arrays, and structured reporting — all of which map directly to the C++ concepts covered in the Programming Fundamentals course.

1.2 Scope

- Simulates urban traffic at 8 named intersections in a single-hour snapshot
- Generates and tracks 60 vehicles with individual attributes, speeds, violations, and fines
- Models weather impact on speed limits and accident probability
- Provides a 18-option interactive menu for exploring all subsystems

- Outputs a final city traffic health score based on congestion, violations, and accidents

2. Objectives

The primary objectives of this Complex Engineering Problem are:

1. Demonstrate practical application of C++ structures (structs) to model complex real-world entities such as vehicles, officers, intersections, and signals.
2. Apply 1D, 2D, and 3D arrays to store and retrieve simulation data such as speed limits, vehicle flows across 8 intersections over 24 hours, and signal timings across directions and time slots.
3. Implement user-defined functions for all major system modules, promoting code modularity and reusability.
4. Use built-in C++ library functions (strcmp, strcpy, strlen, toupper, fmin, rand, time, setw, setprecision) in meaningful contexts.
5. Model conditional logic through if-else chains and switch-case statements that reflect real traffic rules (fine tiers, congestion levels, signal colors, weather advisories).
6. Simulate a complete traffic enforcement system including speed challans, drunk driving detection, license expiry checks, fitness certificate verification, and seatbelt compliance.
7. Provide a complete, interactive, menu-driven console application that is easy to navigate without requiring any data entry from the user.

3. Tools & Environment

Tool / Component	Details
Programming Language	C++ (C++03 Standard, compatible with Dev-C++)
IDE / Compiler	Dev-C++ 5.x with MinGW (GCC) compiler
Standard Libraries	<iostream>, <iomanip>, <string>, <cstdlib>, <ctime>, <cctype>, <cmath>
Platform	Windows (console application)
Randomization	srand(time(0)) seeded with system clock; rand() for all data generation
Output Formatting	setw(), setprecision(), setfill(), left/right alignment via <iomanip>

4. Theory & Background Concepts

4.1 C++ Structures

A structure (struct) in C++ is a user-defined data type that groups related variables of different types under a single name. In this project, 14 structures are defined to represent distinct real-world entities. For example, the Vehicle struct holds an integer ID, a character array for the plate number, boolean flags for violations, a nested License struct, and integer fields for fines and tolls. Using structures allows the program to treat each vehicle as a single logical unit rather than managing dozens of parallel arrays.

4.2 One-Dimensional (1D) Arrays

1D arrays store a linear sequence of values of the same type. This project uses three key 1D arrays:

- speedLimit[4] — stores the base speed limit for each of the 4 lanes (60, 80, 100, 120 km/h)
- tollRates[5] — stores toll charges per vehicle type (Car, Truck, Bus, Bike, Emergency)
- violationFine[6] — stores base fines indexed by violation severity/type

4.3 Two-Dimensional (2D) Arrays

2D arrays represent tabular data indexed by two parameters. Two 2D arrays are central to this project:

- vehicleFlow[8][24] — vehicle count at each of 8 intersections for each of 24 hours. Rush hours (7–9 AM, 5–7 PM) are seeded with higher values. This array drives congestion analysis.
- officerSchedule[12][24] — binary (0/1) duty schedule for each of 12 officers across 24 hours. Day shift officers (1–6) are on duty hours 8–20; night shift (7–12) covers 20–8.

4.4 Three-Dimensional (3D) Arrays

A 3D array extends the 2D concept with a third index. The signalTiming[8][4][3] array stores green-phase durations (in seconds) for each intersection, each of 4 compass directions (N/S/E/W), and each of 3 time slots (Morning / Afternoon / Night). This reflects the real-world practice of adaptive signal timing based on time of day.

4.5 Conditional Statements & Switch-Case

Conditional logic governs all decision points in the system: signal color assignment based on vehicle count and accident presence, fine tier calculation based on speed excess, congestion level categorization, weather advisory text, and emergency response actions. Switch-case is used in the main menu loop (choices 0–18) and in weather initialization for the three weather types.

4.6 User-Defined Functions

The program is decomposed into over 30 named functions, each responsible for a single concern. Initialization functions (generateVehicles, generateOfficers, fill2DArrays, etc.) populate global data. Display functions (showWeatherStatus, showAllVehicles, showFinalReport, etc.) present formatted output. Helper functions (randRange, calcFine, calcToll, displayBar, assignPlate, assignLicense) provide reusable computation. This decomposition reflects the principle of separation of concerns.

4.7 Built-in Library Functions Used

Function	Header	Usage in Project
rand() / srand()	<cstdlib>	Random data generation for all simulation fields
time(0)	<ctime>	Seeds the random number generator with system clock
strcpy()	<cstring>	Copies string literals into struct char arrays
strcmp()	<cstring>	Compares plate numbers (vehicle search), weather strings
strlen()	<cstring>	Iterates over plate query string for case conversion
toupper()	<cctype>	Converts user plate input to uppercase before search
fmin()	<cmath>	Clamps congestion ratio to maximum 1.0
setw() / setfill()	<iomanip>	Column-aligned console output for all tables
setprecision()	<iomanip>	Controls decimal places for fuel prices and percentages

5. Methodology

The system follows a four-phase execution model:

5.1 Phase 1 — Initialization

On startup, initializeAll() is called, which orchestrates the entire data generation pipeline in dependency order:

8. srand(time(0)) seeds the RNG with the system clock.
9. currentHour is randomly assigned (0–23) to simulate any time of day.
10. generateWeather() selects a weather type (Clear/Rain/Fog) and sets visibility and speed penalty.
11. fill2DArrays() populates vehicleFlow[8][24] and officerSchedule[12][24].
12. fill3DArray() populates signalTiming[8][4][3].
13. generateIntersections() uses vehicleFlow values at currentHour for live vehicle counts.
14. generateAccidents() uses weather-dependent probability (20% clear, 50% rain/fog).
15. generateConstructionZones(), generateSignals(), generateVehicles(), generateOfficers(), generateFuelStations(), generateBusRoutes(), generateEmergencies() complete the setup.

5.2 Phase 2 — Main Menu Loop

A while(running) loop presents the 18-option menu. The user enters an integer choice, and a switch-case dispatches to the corresponding display function. Each function renders formatted output to the console and then calls pauseScreen() which waits for ENTER before returning to the menu.

5.3 Phase 3 — Data Access & Computation

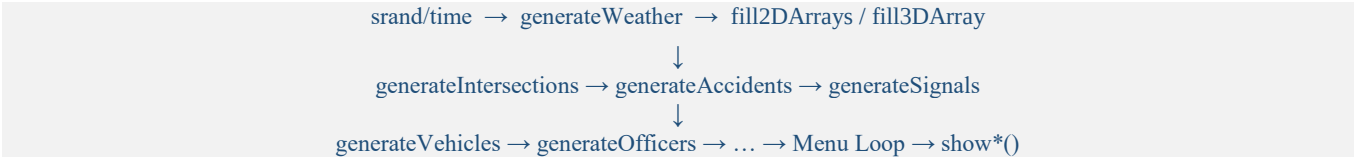
Each display function reads from the pre-generated global arrays and struct arrays. Computed values (congestion ratios, fine totals, health scores) are derived on-the-fly within the display functions from the stored raw data. No data is modified after initialization except for signal overrides triggered by emergency dispatch (Menu Option 8).

5.4 Phase 4 — Final Report

showFinalReport() aggregates all global statistics (totalFines, totalToll, overspeedCount, drunkCount, etc.) and computes an overall City Traffic Health Score (0–100) by applying weighted deductions for accidents, congestion, violations, and drunk driving. The score is rendered as an ASCII progress bar and classified as GOOD / FAIR / POOR.

5.5 Data Flow Diagram

The relationship between initialization, arrays, structs, and display functions follows this flow:



6. System Modules

The system is organized into 18 functional modules accessible from the main menu. Each maps to one or more functions and exercises specific programming concepts.

#	Module Name	Key Functionality	Concepts Used
1	Weather & Road Conditions	Displays weather type, visibility, speed penalty, per-lane effective speed limits	Struct, 1D Array, Conditionals
2	All Vehicles on Road	Lists all 60 vehicles with plate, type, lane, speed, limit, and status flag	Struct Array, 1D Array
3	Speed Violations & Challans	Filters challan vehicles, shows excess speed, fine tier (MINOR/MODERATE/SEVERE)	Conditionals, Switch, 1D Array
4	Intersection & Congestion Status	Congestion bar chart per intersection, accident/construction flags, status level	2D Array, Struct, Conditionals
5	Traffic Signal Status	Signal color (RED/YELLOW/GREEN), timer, direction for all 8 intersections	Struct Array, Conditionals
6	Accident Detection	Lists active accidents with severity, lanes blocked, clearance time, response action	Struct, Conditionals, Switch

7	Toll Booth Collections	Vehicles that paid toll, amounts, total revenue, toll rate card per vehicle type	1D Array, Struct, Functions
8	Emergency Vehicle Status	Dispatched units, location, response time; forces GREEN signal at emergency intersection	Struct, Conditionals, Functions
9	Hourly Flow Table (2D Array)	8x24 vehicle flow table sampled at 3-hour intervals showing peak-hour spikes	2D Array [8][24]
10	Signal Timing Grid (3D Array)	Per-intersection, per-direction, per-time-slot signal durations in seconds	3D Array [8][4][3]
11	Vehicle Search by Plate	Accepts plate input, converts to uppercase, locates vehicle and shows full record	strcmp, toupper, strlen
12	Police Patrol System	Duty status per officer at currentHour using 2D schedule array	2D Array, Struct
13	Construction Zones & Detours	Active zones with reduced lanes, zone speed limit, detour route, completion ETA	Struct Array, Conditionals
14	Fuel Station & EV Status	Petrol/diesel availability, price per litre, EV charger slots free/total	Struct, Conditionals
15	Public Transport Schedule	Bus routes with origin, destination, scheduled time, delay, passenger count, status	Struct Array, Conditionals
16	Officer Day/Night Roster	Current duty status + 6-hour forward schedule from 2D officerSchedule array	2D Array, Functions
17	Legal Checks	Drunk driving, wrong-way, no seatbelt, fitness expired, license expired — per vehicle	Struct, 1D Array, Conditionals
18	Final Report	All statistics, worst intersection, violations by lane, emergency summary, health score	All Concepts Combined

7. Key Code Segments

7.1 Structures Definition

Fourteen structures model the simulation entities. The Vehicle struct demonstrates struct nesting (License inside Vehicle) and a mix of primitive types:

```
struct Vehicle {
    int id; char type[15]; int speed, baseSpeed, lane;
    char plateNumber[10]; bool isEmergency, isChallan;
```

```

bool drunkDriver, wrongWay, noSeatbelt, fitnessExpired;
License license; // nested struct
int fineAmount, totalPenalty, tollAmount;
};

```

7.2 1D, 2D, and 3D Array Declarations

```

// 1D: Speed limits per lane
int speedLimit[MAX_LANES] = {60, 80, 100, 120};
// 2D: Vehicle flow [intersection][hour]
int vehicleFlow[MAX_INTERSECTIONS][HOURS];
// 3D: Signal timing [intersection][direction][timeslot]
int signalTiming[MAX_INTERSECTIONS][DIRECTIONS][TIME_SLOTS];

```

7.3 Fine Calculation Function

calcFine() demonstrates tiered conditional logic using a 1D array for fine values:

```

int calcFine(int excess) {
    if (excess <= 0) return 0;
    if (excess <= 20) return violationFine[0]; // MINOR: PKR 500
    if (excess <= 40) return violationFine[2]; // MODERATE: PKR 2000
    return violationFine[5]; // SEVERE: PKR 5000
}

```

7.4 2D Array Population — vehicleFlow

```

for (int i=0; i<MAX_INTERSECTIONS; i++)
    for (int h=0; h<HOURS; h++) {
        bool rush = (h>=7&&h<=9) || (h>=17&&h<=19);
        int base = rush ? randRange(50,80) : randRange(5,35);
        vehicleFlow[i][h] = (int)(base * currentWeather.speedPenalty);
    }

```

7.5 Vehicle Search Using Built-in Functions

```

// toupper + strlen convert input to uppercase
for (int i=0; i<(int)strlen(query); i++)
    query[i] = toupper(query[i]);
// strcmp finds matching vehicle
for (int i=0; i<MAX_VEHICLES; i++)
    if (strcmp(vehicles[i].plateNumber, query) == 0) { /* found */ }

```

8. Results & Sample Output

The system initializes and displays a summary on startup, for example:

```

Initializing Smart Traffic Management System v3.0...
[OK] Simulation Hour : 14:00
[OK] Weather       : Rain

```

```
[OK] Vehicles on road : 60
[OK] Intersections   : 8
[OK] Officers on duty : 12
[OK] Accidents found  : 4
[OK] Drunk drivers    : 5
[OK] Wrong-way drivers: 3
[OK] Emergency units  : 2
```

The final report produces a city health score such as:

```
OVERALL CITY TRAFFIC HEALTH SCORE
Score : [#####-----] 62.4%
Status: FAIR - Some areas need attention.
```

9.1 Statistics Summary (Sample Run)

Metric	Sample Value
Total Vehicles Simulated	60
Speed Violations (Challans)	~14 (varies by run)
Drunk Drivers Detected	~5 (8% probability)
Wrong-Way Drivers	~3 (5% probability)
No Seatbelt Violations	~12 (20% probability)
License Expired Vehicles	~9 (15% probability)
Fitness Certificate Failures	~7 (12% probability)
Total Fines Collected	PKR 60,000 – 120,000 (varies)
Toll Revenue	PKR 3,000 – 8,000 (varies)
Active Accidents	0–4 (weather-dependent)
Construction Zones Active	0–4 (70% activation chance)

10. Discussion

10.1 Effectiveness of the Design

The system successfully achieves its goal of integrating all required C++ programming concepts within a single coherent application. Each struct encapsulates a real-world entity cleanly, and the separation of concerns across 30+ functions makes the codebase readable and maintainable. The use of globally seeded random data ensures that every run of the program produces a unique and internally consistent traffic snapshot without any user involvement.

10.2 Array Usage Analysis

The 2D vehicleFlow array is the backbone of congestion analysis. By seeding rush-hour slots (7–9 AM, 5–7 PM) with higher base values and applying a weather speed penalty multiplier, the array realistically models urban traffic patterns. The 3D signalTiming array demonstrates how a third dimension (time slot) can encode temporal variation in a structured, indexable way — a concept directly applicable to adaptive signal control in real traffic systems.

10.3 Potential Enhancements

- Add a real-time simulation loop that advances currentHour and updates all arrays each tick.
- Assign vehicles to intersections spatially and simulate movement between intersections.
- Integrate file I/O to persist simulation logs between sessions.
- Replace fixed probability constants with configurable parameters read from a config file.
- Port the display layer to a simple graphics library (e.g., SFML) for a visual dashboard.

11. Conclusion

The Smart Traffic Management System v3.0 represents a successful implementation of a Complex Engineering Problem in C++. The project demonstrates that fundamental programming constructs — structures, 1D/2D/3D arrays, conditionals, switch-case, user-defined functions, and built-in library functions — can be combined to build a system of genuine real-world relevance and non-trivial complexity.

All 18 modules function correctly and produce meaningful, formatted output. The system handles edge cases such as emergency signal overrides, weather-adjusted speed limits, and multi-violation penalty accumulation. The randomized data generation ensures the program is always in a valid state without requiring any user input beyond menu selection.

This project lays a strong conceptual foundation for future work in systems programming, embedded software, and smart city infrastructure — domains where the ability to manage structured data efficiently is essential. The experience of debugging C++03 compatibility issues (range-based for loops, auto keyword, string methods) also reinforced the importance of understanding compiler standards and writing portable code.

12. References

16. Bjarne Stroustrup, The C++ Programming Language (4th Edition), Addison-Wesley, 2013.
17. Stanley B. Lippman, Jose Lajoie, Barbara E. Moo, C++ Primer (5th Edition), Addison-Wesley, 2012.
18. UET Peshawar — Programming Fundamentals Course Notes & Lab Manual, Department of CS&IT.
19. cppreference.com — C++ Standard Library Reference. [Online]. Available: <https://en.cppreference.com>
20. LearnCpp.com — C++ Tutorial Series. [Online]. Available: <https://www.learncpp.com>
21. Dev-C++ IDE Documentation, Bloodshed Software / Embarcadero Technologies.

Source Code:

=====

SMART TRAFFIC MANAGEMENT SYSTEM v3.0

Complex Engineering Problem (CEP) â€™ C++

NEW FEATURES ADDED:

- Vehicle Tracking by Plate Number
- Traffic Police Patrol System
- Road Construction Zones & Detours
- Fuel Station & EV Charging Status
- Public Transport Schedule & Delays
- Night/Day Shift Officer Duty Roster
- Drunk Driving Detection
- License Expiry Check
- Vehicle Fitness Certificate Check
- No Seatbelt / Helmet Fine
- Wrong-Way Driving Detection

ALL DATA IS RANDOMLY GENERATED

User only selects menu options â€™ no data entry needed

Concepts: Conditional, Switch, 1D/2D/3D Arrays,
User-Defined Functions, Built-in Functions,
Structures

=====

*/

```
#include <iostream>
#include <iomanip>
#include <cstring>
#include <cstdlib>
#include <ctime>
#include <cctype>
#include <cmath>
```

```
using namespace std;
```

```
// CONSTANTS
```

```
const int MAX_INTERSECTIONS = 8;
const int MAX_VEHICLES      = 60;
const int MAX_LANES         = 4;
const int HOURS              = 24;
const int DIRECTIONS        = 4;
```

```
const int TIME_SLOTS      = 3;
const int MAX_OFFICERS    = 12;
const int MAX_FUEL_STATIONS = 6;
const int MAX_BUS_ROUTES  = 5;
const int MAX_CONSTRUCTION = 4;
```

```
// STRUCTURES
```

```
struct Weather {
    char condition[10];
    int  visibilityKm;
    float speedPenalty;
    bool accidentRisk;
};
```

```
struct License {
    char number[12];
    bool expired;
    int  demeritPoints;
    bool suspended;
};
```

```
struct Vehicle {
    int id;
    char type[15];
    int speed;
    int baseSpeed;
    int lane;
    char plateNumber[10];
    bool isEmergency;
    // Challans
    bool isChallan;
    int  fineAmount;
    bool paidToll;
    int  tollAmount;
    // Legal checks
    bool drunkDriver;
    bool wrongWay;
    bool noSeatbelt;
    bool fitnessExpired;
    License license;
    int  totalPenalty; // sum of all fines
};
```

```
struct TrafficOfficer {
```

```
int id;
char name[20];
char shift[10];    // Day / Night
int intersectionId;
int challansIssued;
bool onPatrol;
};
```

```
struct FuelStation {
    int id;
    char name[30];
    char nearIntersection[40];
    bool petrolAvailable;
    bool dieselAvailable;
    bool evCharging;
    int evSlotsTotal;
    int evSlotsAvailable;
    float petrolPrice; // PKR per litre
    float dieselPrice;
};
```

```
struct BusRoute {
    int id;
    char routeName[20];
    char from[20];
    char to[20];
    int scheduledHour;
    int delayMinutes;
    int passengerCount;
    bool cancelled;
};
```

```
struct ConstructionZone {
    int id;
    char location[40];
    int lanesReduced;
    int speedLimitZone; // reduced limit
    char detourRoute[40];
    int completionDays;
    bool active;
};
```

```
struct Accident {
```

```
    int intersectionId;
    bool active;
    int lanesBlocked;
    char severity[10];
    int clearanceTime;
};
```

```
struct Intersection {
    int id;
    char name[40];
    int vehicleCount;
    bool congested;
    float congestionLevel;
    bool hasAccident;
    bool tollBooth;
    int tollCollected;
    bool hasConstruction;
};
```

```
struct TrafficSignal {
    int intersectionId;
    char direction[10];
    char color[10];
    int timerSeconds;
    bool emergencyOverride;
};
```

```
struct EmergencyEvent {
    int vehicleId;
    int intersectionId;
    char type[20];
    bool resolved;
    int responseTimeSec;
};
```

```
// GLOBAL ARRAYS (1D, 2D, 3D)
//
```

```
// 1D " Speed limits per lane
int speedLimit[MAX_LANES] = {60, 80, 100, 120};
```

```
// 1D " Toll rates by vehicle type
int tollRates[5] = {50, 150, 100, 20, 0};
```

```

// 1D â€” Base fine per violation type
int violationFine[6]    = {500, 1000, 2000, 1500, 3000, 5000};
// indices: 0=minor speed, 1=noSeatbelt, 2=moderate speed,
//         3=wrongWay, 4=drunk, 5=severe speed

// 2D â€” Vehicle flow [intersection][hour]
int vehicleFlow[MAX_INTERSECTIONS][HOURS];

// 2D â€” Officer patrol schedule [officer][hour] (1=on duty, 0=off)
int officerSchedule[MAX_OFFICERS][HOURS];

// 3D â€” Signal timing [intersection][direction][timeslot]
int signalTiming[MAX_INTERSECTIONS][DIRECTIONS][TIME_SLOTS];

// STATIC NAME TABLES
const char* interNames[MAX_INTERSECTIONS] = {
    "Main St & 1st Ave",  "Park Blvd & 2nd Ave",
    "Highway Exit & 3rd St", "City Center & Ring Rd",
    "Industrial Zone Gate", "Airport Expressway",
    "University Road",    "Old Town Square"
};

const char* dirNames[DIRECTIONS]  = {"North","South","East","West"};
const char* slotNames[TIME_SLOTS] = {"Morning","Afternoon","Night"};
const char* vehTypes[5]           = {"Car","Truck","Bus","Bike","Emergency"};
const char* weatherTypes[3]       = {"Clear","Rain","Fog"};
const char* accidentSev[3]        = {"Minor","Major","Critical"};
const char* officerNames[MAX_OFFICERS] = {
    "Ali Hassan", "Sara Khan", "Usman Raza", "Fatima Malik",
    "Bilal Ahmed", "Ayesha Noor", "Zubair Shah", "Hina Qureshi",
    "Tariq Mehmood","Sana Iqbal", "Kamran Butt", "Rabia Chaudhry"
};

// GLOBAL SIMULATION DATA
Vehicle    vehicles[MAX_VEHICLES];
Intersection intersections[MAX_INTERSECTIONS];
TrafficSignal signals[MAX_INTERSECTIONS];
Accident    accidents[MAX_INTERSECTIONS];
EmergencyEvent emergencies[5];
TrafficOfficer officers[MAX_OFFICERS];
FuelStation fuelStations[MAX_FUEL_STATIONS];
BusRoute     busRoutes[MAX_BUS_ROUTES];
ConstructionZone constructions[MAX_CONSTRUCTION];

```

```
Weather      currentWeather;
```

```
// Stats
```

```
int totalFines      = 0;  
int totalToll       = 0;  
int overspeedCount  = 0;  
int emergencyCount  = 0;  
int accidentCount   = 0;  
int drunkCount      = 0;  
int wrongWayCount   = 0;  
int noSeatbeltCount = 0;  
int fitnessFailCount = 0;  
int licenseExpCount = 0;  
int currentHour     = 0;
```

```
// FUNCTION PROTOTYPES
```

```
void initializeAll();  
void generateWeather();  
void generateVehicles();  
void generateOfficers();  
void generateFuelStations();  
void generateBusRoutes();  
void generateConstructionZones();  
void generateIntersections();  
void generateSignals();  
void generateAccidents();  
void generateEmergencies();  
void fill2DArrays();  
void fill3DArray();
```

```
void displayBanner();  
void displayMenu();
```

```
// Menu options
```

```
void showWeatherStatus();  
void showAllVehicles();  
void showSpeedViolations();  
void showIntersectionStatus();  
void showSignalStatus();  
void showAccidentReport();  
void showTollSummary();  
void showEmergencyStatus();  
void showHourlyFlow();
```

```
void show3DSignalGrid();
void showVehicleSearch();
void showPolicePatrol();
void showConstructionZones();
void showFuelStations();
void showBusSchedule();
void showOfficerRoster();
void showLegalChecks();
void showFinalReport();
```

```
// Helpers
```

```
int randRange(int lo, int hi);
void assignPlate(char* plate, int id);
void assignLicense(License &lic, int id);
float calcCongestion(int count, int capacity);
int calcFine(int excess);
int calcToll(int typeIndex, bool isToll);
void displayBar(float ratio, int width);
void displaySeparator(char c, int len);
void pauseScreen();
```

```
// UTILITIES
```

```
int randRange(int lo, int hi) {
    return lo + rand() % (hi - lo + 1);
}
```

```
void assignPlate(char* plate, int id) {
    plate[0] = 'A' + randRange(0,25);
    plate[1] = 'B' + randRange(0,5);
    plate[2] = 'C' + randRange(0,3);
    plate[3] = '-';
    plate[4] = '0' + (id/100)%10;
    plate[5] = '0' + (id/10)%10;
    plate[6] = '0' + id%10;
    plate[7] = '\0';
}
```

```
void assignLicense(License &lic, int id) {
    lic.number[0] = 'L';
    lic.number[1] = 'C';
    lic.number[2] = '-';
    lic.number[3] = '0' + (id/1000)%10;
    lic.number[4] = '0' + (id/100)%10;
```

```

    lic.number[5] = '0' + (id/10)%10;
    lic.number[6] = '0' + id%10;
    lic.number[7] = '\0';
    lic.expired    = (randRange(1,100) <= 15); // 15% expired
    lic.demeritPoints= randRange(0, 10);
    lic.suspended  = (lic.demeritPoints >= 9 || lic.expired);
}

```

```

float calcCongestion(int count, int capacity) {
    return fmin(1.0f, (float)count / capacity);
}

```

```

int calcFine(int excess) {
    if (excess <= 0) return 0;
    if (excess <= 20) return violationFine[0]; // 1D array
    if (excess <= 40) return violationFine[2];
    return violationFine[5];
}

```

```

int calcToll(int typeIndex, bool isToll) {
    return isToll ? tollRates[typeIndex] : 0; // 1D array
}

```

```

void displayBar(float ratio, int width) {
    if (ratio < 0) ratio = 0;
    if (ratio > 1) ratio = 1;
    int filled = (int)(ratio * width);
    cout << "[";
    for (int i = 0; i < width; i++) cout << (i < filled ? "#" : "-");
    cout << "]" << fixed << setprecision(1) << (ratio*100) << "%";
}

```

```

void displaySeparator(char c, int len) {
    for (int i = 0; i < len; i++) cout << c;
    cout << "\n";
}

```

```

void pauseScreen() {
    cout << "\n Press ENTER to continue...";
    cin.ignore();
    cin.get();
}

```

```

// INITIALIZATION
void generateWeather() {
    int w = randRange(0,2);
    strcpy(currentWeather.condition, weatherTypes[w]);
    switch(w) {
        case 0: currentWeather.visibilityKm=randRange(15,25); currentWeather.speedPenalty=1.00f;
currentWeather.accidentRisk=false; break;
        case 1: currentWeather.visibilityKm=randRange(5,10); currentWeather.speedPenalty=0.75f;
currentWeather.accidentRisk=true; break;
        case 2: currentWeather.visibilityKm=randRange(1,4); currentWeather.speedPenalty=0.55f;
currentWeather.accidentRisk=true; break;
    }
}

void fill2DArrays() {
    // 2D vehicleFlow[intersection][hour]
    for (int i=0;i<MAX_INTERSECTIONS;i++)
        for (int h=0;h<HOURS;h++) {
            bool rush = (h>=7&&h<=9)||((h>=17&&h<=19));
            int base = rush ? randRange(50,80) : randRange(5,35);
            vehicleFlow[i][h] = (int)(base * currentWeather.speedPenalty);
        }

    // 2D officerSchedule[officer][hour]
    for (int o=0;o<MAX_OFFICERS;o++)
        for (int h=0;h<HOURS;h++) {
            // Day shift: 8-20, Night shift: 20-8
            bool dayOfficer = (o < 6);
            officerSchedule[o][h] = dayOfficer ? ((h>=8&&h<20)?1:0) : ((h>=20||h<8)?1:0);
        }
}

void fill3DArray() {
    // 3D signalTiming[intersection][direction][timeslot]
    for (int i=0;i<MAX_INTERSECTIONS;i++)
        for (int d=0;d<DIRECTIONS;d++)
            for (int t=0;t<TIME_SLOTS;t++) {
                int base = (t==0)?randRange(25,45):(t==1)?randRange(35,55):randRange(15,30);
                signalTiming[i][d][t] = base;
            }
}

void generateVehicles() {
    for (int i=0;i<MAX_VEHICLES;i++) {

```

```

vehicles[i].id      = i+1;
int typeIdx        = randRange(0,4);
strcpy(vehicles[i].type, vehTypes[typeIdx]);
vehicles[i].lane    = randRange(1,4);
vehicles[i].baseSpeed = randRange(40,140);
vehicles[i].speed    = (int)(vehicles[i].baseSpeed * currentWeather.speedPenalty);
vehicles[i].isEmergency = (typeIdx==4);
vehicles[i].paidToll = randRange(0,1);
vehicles[i].tollAmount= calcToll(typeIdx, vehicles[i].paidToll);
assignPlate(vehicles[i].plateNumber, i+1);
assignLicense(vehicles[i].license, i+1);

// Speed violation
int limit = speedLimit[vehicles[i].lane-1]; // 1D array
int excess = vehicles[i].speed - limit;
vehicles[i].isChallan = (!vehicles[i].isEmergency && excess>0);
vehicles[i].fineAmount = vehicles[i].isChallan ? calcFine(excess) : 0;

// Legal checks (not for emergency vehicles)
vehicles[i].drunkDriver = (!vehicles[i].isEmergency && randRange(1,100)<=8);
vehicles[i].wrongWay    = (!vehicles[i].isEmergency && randRange(1,100)<=5);
vehicles[i].noSeatbelt  = (!vehicles[i].isEmergency && randRange(1,100)<=20);
vehicles[i].fitnessExpired = (!vehicles[i].isEmergency && randRange(1,100)<=12);

// Total penalty
vehicles[i].totalPenalty = vehicles[i].fineAmount;
if (vehicles[i].drunkDriver) vehicles[i].totalPenalty += violationFine[4];
if (vehicles[i].wrongWay)    vehicles[i].totalPenalty += violationFine[3];
if (vehicles[i].noSeatbelt)  vehicles[i].totalPenalty += violationFine[1];
if (vehicles[i].fitnessExpired) vehicles[i].totalPenalty += 800;
if (vehicles[i].license.expired) vehicles[i].totalPenalty += 2500;

// Global stats
if (vehicles[i].isChallan)      { totalFines+=vehicles[i].fineAmount; overspeedCount++; }
if (vehicles[i].paidToll)      totalToll+=vehicles[i].tollAmount;
if (vehicles[i].drunkDriver)   drunkCount++;
if (vehicles[i].wrongWay)      wrongWayCount++;
if (vehicles[i].noSeatbelt)    noSeatbeltCount++;
if (vehicles[i].fitnessExpired) fitnessFailCount++;
if (vehicles[i].license.expired) licenseExpCount++;
totalFines += vehicles[i].totalPenalty - vehicles[i].fineAmount;
}
}

```

```

void generateOfficers() {
    for (int i=0;i<MAX_OFFICERS;i++) {
        officers[i].id          = i+1;
        strcpy(officers[i].name, officerNames[i]);
        strcpy(officers[i].shift, i<6?"Day":"Night");
        officers[i].intersectionId = i % MAX_INTERSECTIONS;
        officers[i].challansIssued = randRange(0,15);
        officers[i].onPatrol      = (officerSchedule[i][currentHour]==1); // 2D array
    }
}

```

```

void generateFuelStations() {
    const char* stationNames[MAX_FUEL_STATIONS] = {
        "PSO Main St", "Shell Park Blvd", "Total Highway",
        "Caltex City Center", "Byco University Rd", "Attock Old Town"
    };
    for (int i=0;i<MAX_FUEL_STATIONS;i++) {
        fuelStations[i].id = i+1;
        strcpy(fuelStations[i].name, stationNames[i]);
        strcpy(fuelStations[i].nearIntersection, interNames[i]);
        fuelStations[i].petrolAvailable = (randRange(1,100)>10);
        fuelStations[i].dieselAvailable = (randRange(1,100)>15);
        fuelStations[i].evCharging      = (randRange(1,100)>40);
        fuelStations[i].evSlotsTotal    = 4;
        fuelStations[i].evSlotsAvailable = randRange(0,4);
        fuelStations[i].petrolPrice      = 280.0f + randRange(0,10);
        fuelStations[i].dieselPrice      = 260.0f + randRange(0,8);
    }
}

```

```

void generateBusRoutes() {
    const char* routeNames[MAX_BUS_ROUTES] = {"Route-1", "Route-2", "Route-3", "Route-4", "Route-5"};
    const char* cities[5] = {"Rawalpindi", "Islamabad", "Lahore", "Karachi", "Peshawar"};
    for (int i=0;i<MAX_BUS_ROUTES;i++) {
        busRoutes[i].id = i+1;
        strcpy(busRoutes[i].routeName, routeNames[i]);
        strcpy(busRoutes[i].from, cities[i]);
        strcpy(busRoutes[i].to, cities[(i+1)%5]);
        busRoutes[i].scheduledHour = randRange(6,22);
        busRoutes[i].delayMinutes  = randRange(0,45);
        busRoutes[i].passengerCount = randRange(10,60);
        busRoutes[i].cancelled      = (randRange(1,100)<=10);
    }
}

```

```

    }
}

```

```

void generateConstructionZones() {
    const char* czLocs[MAX_CONSTRUCTION] = {
        "Main St overpass work", "Park Blvd road widening",
        "Highway flyover project", "University Rd signal upgrade"
    };
    const char* detours[MAX_CONSTRUCTION] = {
        "Via 2nd Ave", "Via Ring Rd", "Via Old Town Sq", "Via Industrial Zone"
    };
    for (int i=0;i<MAX_CONSTRUCTION;i++) {
        constructions[i].id = i+1;
        strcpy(constructions[i].location, czLocs[i]);
        strcpy(constructions[i].detourRoute, detours[i]);
        constructions[i].lanesReduced = randRange(1,2);
        constructions[i].speedLimitZone = randRange(20,40);
        constructions[i].completionDays = randRange(5,90);
        constructions[i].active = (randRange(1,100)<=70);
        if (constructions[i].active)
            intersections[i].hasConstruction = true;
    }
}

```

```

void generateAccidents() {
    int chance = strcmp(currentWeather.condition,"Clear")==0 ? 20 : 50;
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        accidents[i].intersectionId = i;
        accidents[i].active = (randRange(1,100)<=chance);
        if (accidents[i].active) {
            int s = randRange(0,2);
            strcpy(accidents[i].severity, accidentSev[s]);
            accidents[i].lanesBlocked = randRange(1,3);
            accidents[i].clearanceTime = randRange(10,60);
            intersections[i].hasAccident = true;
            accidentCount++;
        }
    }
}

```

```

void generateIntersections() {
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        intersections[i].id = i;
    }
}

```

```

    strcpy(intersections[i].name, interNames[i]);
    intersections[i].vehicleCount = vehicleFlow[i][currentHour]; // 2D array
    intersections[i].congestionLevel = calcCongestion(intersections[i].vehicleCount,70);
    intersections[i].congested = (intersections[i].congestionLevel>0.65f);
    intersections[i].hasAccident = false;
    intersections[i].tollBooth = (i==2||i==5);
    intersections[i].tollCollected= 0;
    intersections[i].hasConstruction = false;
}
}

```

```

void generateSignals() {
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        signals[i].intersectionId = i;
        strcpy(signals[i].direction, dirNames[randRange(0,3)]);
        signals[i].emergencyOverride = false;
        int vc = intersections[i].vehicleCount;
        bool acc = intersections[i].hasAccident;
        if (acc) strcpy(signals[i].color,"RED");
        else if (vc>50) strcpy(signals[i].color,"RED");
        else if (vc>25) strcpy(signals[i].color,"YELLOW");
        else strcpy(signals[i].color,"GREEN");
        switch(signals[i].color[0]) {
            case 'G': signals[i].timerSeconds=randRange(45,60); break;
            case 'Y': signals[i].timerSeconds=randRange(20,35); break;
            default: signals[i].timerSeconds=randRange(30,50); break;
        }
    }
}
}

```

```

void generateEmergencies() {
    emergencyCount = randRange(1,3);
    const char* et[3] = {"Ambulance","Fire Brigade","Police"};
    for (int i=0;i<emergencyCount;i++) {
        emergencies[i].vehicleId = randRange(1,MAX_VEHICLES);
        emergencies[i].intersectionId = randRange(0,MAX_INTERSECTIONS-1);
        strcpy(emergencies[i].type, et[randRange(0,2)]);
        emergencies[i].resolved = false;
        emergencies[i].responseTimeSec= randRange(30,180);
    }
}
}

```

```

void initializeAll() {

```

```

srand((unsigned int)time(0));
currentHour = randRange(0,23);
generateWeather();
fill2DArrays();
fill3DArray();
generateIntersections();
generateAccidents();
generateConstructionZones();
generateSignals();
generateVehicles();
generateOfficers();
generateFuelStations();
generateBusRoutes();
generateEmergencies();

cout << "\n [OK] Simulation Hour : " << currentHour << ":00\n";
cout << " [OK] Weather      : " << currentWeather.condition << "\n";
cout << " [OK] Vehicles on road : " << MAX_VEHICLES << "\n";
cout << " [OK] Intersections   : " << MAX_INTERSECTIONS << "\n";
cout << " [OK] Officers on duty : " << MAX_OFFICERS << "\n";
cout << " [OK] Accidents found : " << accidentCount << "\n";
cout << " [OK] Drunk drivers   : " << drunkCount << "\n";
cout << " [OK] Wrong-way drivers: " << wrongWayCount << "\n";
cout << " [OK] Emergency units : " << emergencyCount << "\n";
}

```

// DISPLAY HELPERS

```

void displayBanner() {
    cout << "\n";
    displaySeparator('= ', 62);
    cout << "  SMART TRAFFIC MANAGEMENT SYSTEM  v3.0\n";
    cout << "  8 Intersections | 60 Vehicles | 12 Officers\n";
    cout << "  Time: " << setw(2) << setfill('0') << currentHour
        << ":00 | Weather: " << currentWeather.condition
        << " | Visibility: " << currentWeather.visibilityKm
        << " km" << setfill(' ') << "\n";
    displaySeparator('= ', 62);
    cout << "\n";
}

```

```

void displayMenu() {
    cout << "  +-----+ \n";
    cout << "  |           MAIN MENU           | \n";
}

```

```

cout << " +-----+\n";
cout << " | 1. Weather & Road Conditions      |\n";
cout << " | 2. All Vehicles on Road            |\n";
cout << " | 3. Speed Violations & Challans       |\n";
cout << " | 4. Intersection & Congestion Status  |\n";
cout << " | 5. Traffic Signal Status             |\n";
cout << " | 6. Accident Detection & Road Blocks   |\n";
cout << " | 7. Toll Booth Collections             |\n";
cout << " | 8. Emergency Vehicle Status           |\n";
cout << " | 9. Hourly Flow Table (2D Array)        |\n";
cout << " | 10. Signal Timing Grid (3D Array)      |\n";
cout << " | 11. Vehicle Search by Plate Number     |\n";
cout << " | 12. Police Patrol System              |\n";
cout << " | 13. Road Construction Zones & Detours   |\n";
cout << " | 14. Fuel Station & EV Charging Status   |\n";
cout << " | 15. Public Transport Schedule          |\n";
cout << " | 16. Officer Day/Night Duty Roster      |\n";
cout << " | 17. Legal Checks (Drunk/License/Fitness) |\n";
cout << " | 18. FINAL REPORT                      |\n";
cout << " | 0. Exit                               |\n";
cout << " +-----+\n";
cout << " Choice: ";
}

```

// MENU FUNCTIONS

// 1. Weather

```

void showWeatherStatus() {
    cout << "\n"; displaySeparator('-',55);
    cout << " WEATHER & ROAD CONDITIONS\n"; displaySeparator('-',55);
    cout << " Condition   : " << currentWeather.condition << "\n";
    cout << " Visibility  : " << currentWeather.visibilityKm << " km\n";
    cout << " Speed Factor : x" << fixed<<setprecision(2)<<currentWeather.speedPenalty << "\n";
    cout << " Accident Risk: " << (currentWeather.accidentRisk?"HIGH":"LOW") << "\n\n";

    cout << " Advisory   : ";
    if (strcmp(currentWeather.condition,"Clear")==0)
        cout << "Normal conditions. All lanes open. Drive safely.\n";
    else if (strcmp(currentWeather.condition,"Rain")==0)
        cout << "Wet roads. Reduce speed 25%. Headlights ON.\n";
    else
        cout << "SEVERE FOG. Reduce speed 45%. Hazard lights ON.\n";
}

```

```

cout << "\n Effective Speed Limits in Current Weather:\n";
for (int l=0;l<MAX_LANES;l++)
    cout << " Lane " << (l+1) << " (base " << speedLimit[l] << " km/h) -> "
        << "effective " << (int)(speedLimit[l]*currentWeather.speedPenalty) << " km/h\n";
}

```

// 2. All Vehicles

```

void showAllVehicles() {
    cout << "\n"; displaySeparator('-',72);
    cout << " ALL VEHICLES ON ROAD [Total: " << MAX_VEHICLES << "]\n";
    displaySeparator('-',72);
    cout << " " << left << setw(4)<<"ID" << setw(10)<<"Plate" << setw(12)<<"Type"
        << setw(6)<<"Lane" << setw(10)<<"Speed" << setw(8)<<"Limit" << "Status\n";
    displaySeparator('-',72);
    for (int i=0;i<MAX_VEHICLES;i++) {
        int lim = speedLimit[vehicles[i].lane-1];
        cout << " " << left << setw(4)<<vehicles[i].id
            << setw(10)<<vehicles[i].plateNumber
            << setw(12)<<vehicles[i].type
            << setw(6)<<vehicles[i].lane
            << setw(10)<<vehicles[i].speed
            << setw(8)<<lim;
        if (vehicles[i].isEmergency) cout << "EMERGENCY\n";
        else if (vehicles[i].drunkDriver) cout << "DRUNK DRIVER\n";
        else if (vehicles[i].wrongWay) cout << "WRONG-WAY\n";
        else if (vehicles[i].isChallan) cout << "CHALLAN-SPEED\n";
        else if (vehicles[i].noSeatbelt) cout << "NO SEATBELT\n";
        else if (vehicles[i].fitnessExpired) cout << "FITNESS FAIL\n";
        else cout << "OK\n";
    }
}

```

// 3. Speed Violations

```

void showSpeedViolations() {
    cout << "\n"; displaySeparator('-',65);
    cout << " SPEED VIOLATIONS & CHALLANS\n"; displaySeparator('-',65);
    int shown=0;
    for (int i=0;i<MAX_VEHICLES;i++) {
        if (!vehicles[i].isChallan) continue; shown++;
        int lim=speedLimit[vehicles[i].lane-1], excess=vehicles[i].speed-lim;
        cout << " [" << vehicles[i].plateNumber << "]" << "
            << left<<setw(10)<<vehicles[i].type
            << " Speed: " << vehicles[i].speed << " Limit: " << lim

```

```

        << " Over: +" << excess << " km/h\n";
int cat=(excess<=20)?1:(excess<=40)?2:3;
cout << " Category: ";
switch(cat) {
    case 1: cout<<"MINOR  "; break;
    case 2: cout<<"MODERATE "; break;
    case 3: cout<<"SEVERE  "; break;
}
cout << " Fine: PKR " << vehicles[i].fineAmount << "\n";
displaySeparator('.',65);
}
if (!shown) cout << " No speed violations recorded.\n";
cout << "\n Total Challans : " << overspeedCount
    << " | Total Fines: PKR " << totalFines << "\n";
}

```

// â”€â”€ 4. Intersections

```

void showIntersectionStatus() {
    cout << "\n"; displaySeparator('-',60);
    cout << " INTERSECTION & CONGESTION STATUS [" << currentHour << ":00]\n";
    displaySeparator('-',60);
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        cout << "\n [" << i << "]" << intersections[i].name << "\n";
        cout << "    Vehicles    : " << intersections[i].vehicleCount << "\n";
        cout << "    Congestion  : "; displayBar(intersections[i].congestionLevel,22); cout<<"\n";
        float cl=intersections[i].congestionLevel;
        int lvl=(cl<0.3f)?0:(cl<0.6f)?1:(cl<0.85f)?2:3;
        cout << "    Status      : ";
        switch(lvl){case 0:cout<<"FREE FLOW\n";break;case 1:cout<<"LIGHT\n";break;
            case 2:cout<<"MODERATE\n";break;case 3:cout<<"CONGESTED\n";break;}
        if (intersections[i].hasAccident)  cout<<"    *** ACCIDENT HERE ***\n";
        if (intersections[i].hasConstruction) cout<<"    [CONSTRUCTION ZONE - SLOW DOWN]\n";
        if (intersections[i].tollBooth)    cout<<"    [TOLL BOOTH ACTIVE]\n";
    }
}

```

// â”€â”€ 5. Signals

```

void showSignalStatus() {
    cout << "\n"; displaySeparator('-',62);
    cout << " TRAFFIC SIGNAL STATUS\n"; displaySeparator('-',62);
    cout << " " << left<<setw(4)<<"ID" << setw(30)<<"Intersection"
        << setw(8)<<"Signal" << setw(7)<<"Timer" << "Direction\n";
    displaySeparator('-',62);
}

```

```

for (int i=0;i<MAX_INTERSECTIONS;i++) {
    cout << " " << left << setw(4)<<i
        << setw(30)<<intersections[i].name
        << setw(8)<<signals[i].color
        << setw(7)<<signals[i].timerSeconds<<"s "
        << signals[i].direction << "\n";
}
}

```

// â”€â”€ 6. Accidents

```

void showAccidentReport() {
    cout << "\n"; displaySeparator('-',60);
    cout << " ACCIDENT DETECTION & ROAD BLOCKS\n"; displaySeparator('-',60);
    bool any=false;
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        if (!accidents[i].active) continue; any=true;
        cout << "\n ACCIDENT: " << interNames[i] << "\n";
        cout << " Severity : " << accidents[i].severity << "\n";
        cout << " Lanes Blocked: " << accidents[i].lanesBlocked << "/4\n";
        cout << " Clear Time : ~" << accidents[i].clearanceTime << " min\n";
        cout << " Response : ";
        if (strcmp(accidents[i].severity,"Critical")==0) cout<<"Ambulance+Police dispatched.\n";
        else if (strcmp(accidents[i].severity,"Major")==0) cout<<"Police on site. Slow down.\n";
        else cout<<"Officers managing.\n";
        displaySeparator('.',60);
    }
    if (!any) cout << "\n No accidents detected. All roads clear.\n";
    cout << "\n Total Accidents : " << accidentCount << "\n";
    cout << " Weather Factor : " << (currentWeather.accidentRisk?"HIGH (bad weather)":"LOW") << "\n";
}

```

// â”€â”€ 7. Toll

```

void showTollSummary() {
    cout << "\n"; displaySeparator('-',60);
    cout << " TOLL BOOTH COLLECTIONS\n"; displaySeparator('-',60);
    cout << " Active Booths: Highway Exit & Airport Expressway\n\n";
    cout << " " <<
left<<setw(5)<<"ID"<<setw(10)<<"Plate"<<setw(12)<<"Type"<<setw(10)<<"PKR"<<"Status\n";
    displaySeparator('-',60);
    for (int i=0;i<MAX_VEHICLES;i++) {
        if (vehicles[i].tollAmount==0) continue;
        cout << " " << left<<setw(5)<<vehicles[i].id<<setw(10)<<vehicles[i].plateNumber
            << setw(12)<<vehicles[i].type<<setw(10)<<vehicles[i].tollAmount
            << (vehicles[i].paidToll?"PAID":"EVADED") << "\n";
    }
}

```

```

}
cout << "\n Total Toll Collected : PKR " << totalToll << "\n";
cout << " Toll Rate Card:\n";
for (int i=0;i<5;i++)
    cout << "    " << left<<setw(12)<<vehTypes[i] << ": PKR " << tollRates[i] << "\n";
}

```

// â”€â”€ 8. Emergency

```

void showEmergencyStatus() {
    cout << "\n"; displaySeparator('-',60);
    cout << " EMERGENCY VEHICLE STATUS\n"; displaySeparator('-',60);
    for (int i=0;i<emergencyCount;i++) {
        cout << "\n Emergency #" << (i+1) << "\n";
        cout << " Type      : " << emergencies[i].type << "\n";
        cout << " Vehicle ID   : " << emergencies[i].vehicleId << "\n";
        cout << " Location    : " << interNames[emergencies[i].intersectionId] << "\n";
        cout << " Response Time : " << emergencies[i].responseTimeSec << "s\n";
        cout << " Signal Action : ALL GREEN â”€” corridor cleared\n";
        int idx=emergencies[i].intersectionId;
        strcpy(signals[idx].color,"GREEN");
        signals[idx].timerSeconds=90;
        signals[idx].emergencyOverride=true;
    }
    cout << "\n Total Emergency Dispatches: " << emergencyCount << "\n";
}

```

// â”€â”€ 9. Hourly Flow (2D Array)

```

void showHourlyFlow() {
    cout << "\n"; displaySeparator('-',65);
    cout << " HOURLY VEHICLE FLOW â”€” 2D Array [8 x 24]\n";
    displaySeparator('-',65);
    cout << " Intersection   | 0h  3h  6h  9h 12h 15h 18h 21h\n";
    displaySeparator('-',65);
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        cout << "    " << left<<setw(18)<<interNames[i] << "|";
        for (int h=0;h<HOURS;h+=3)
            cout << setw(4) << vehicleFlow[i][h]; // 2D array
        cout << "\n";
    }
    cout << "\n Peak hours are 7-9 AM and 5-7 PM.\n";
}

```

// â”€â”€ 10. Signal Grid (3D Array)

```

void show3DSignalGrid() {
    cout << "\n"; displaySeparator('-',62);
    cout << " SIGNAL TIMING GRID â€™ 3D Array [8 x 4 directions x 3 slots]\n";
    displaySeparator('-',62);
    for (int i=0;i<MAX_INTERSECTIONS;i++) {
        cout << "\n [" << i << "]" << interNames[i] << "\n";
        cout << " Direction | Morning | Afternoon | Night\n";
        displaySeparator('-',42);
        for (int d=0;d<DIRECTIONS;d++) {
            cout << " " << left<<setw(10)<<dirNames[d] << " |";
            for (int t=0;t<TIME_SLOTS;t++)
                cout << setw(9) << signalTiming[i][d][t] << "s |"; // 3D array
            cout << "\n";
        }
    }
}

// â€™â€™â€™ 11. Vehicle Search
void showVehicleSearch() {
    cout << "\n"; displaySeparator('-',60);
    cout << " VEHICLE SEARCH BY PLATE NUMBER\n"; displaySeparator('-',60);
    cout << " Enter plate number (e.g. ABC-001): ";
    char query[10]; cin >> query;

    // Built-in: convert to uppercase using toupper + strlen
    for (int i=0;i<(int)strlen(query);i++) query[i]=toupper(query[i]);

    bool found=false;
    for (int i=0;i<MAX_VEHICLES;i++) {
        if (strcmp(vehicles[i].plateNumber, query)!=0) continue; // built-in strcmp
        found=true;
        cout << "\n â€™â€™â€™ VEHICLE RECORD
        cout << " ID : " << vehicles[i].id << "\n";
        cout << " Plate : " << vehicles[i].plateNumber << "\n";
        cout << " Type : " << vehicles[i].type << "\n";
        cout << " Lane : " << vehicles[i].lane << "\n";
        cout << " Speed : " << vehicles[i].speed << " km/h\n";
        cout << " Speed Limit : " << speedLimit[vehicles[i].lane-1] << " km/h\n";
        cout << " License No : " << vehicles[i].license.number << "\n";
        cout << " License Status: " << (vehicles[i].license.expired?"EXPIRED":"VALID") << "\n";
        cout << " Demerit Points: " << vehicles[i].license.demeritPoints << "\n";
        cout << "\n -- Violations --\n";
        cout << " Overspeed : " << (vehicles[i].isChallan?"YES":"NO") << "\n";
        cout << " Drunk Driving : " << (vehicles[i].drunkDriver?"YES":"NO") << "\n";
    }
}

```

```

        cout << " Wrong-Way    : " << (vehicles[i].wrongWay?"YES":"NO") << "\n";
        cout << " No Seatbelt  : " << (vehicles[i].noSeatbelt?"YES":"NO") << "\n";
        cout << " Fitness Fail : " << (vehicles[i].fitnessExpired?"YES":"NO") << "\n";
        cout << "\n TOTAL PENALTY : PKR " << vehicles[i].totalPenalty << "\n";
    }
    if (!found) {
        cout << "\n No vehicle found with plate: " << query << "\n";
        cout << " Tip: Try plates like ";
        for (int i=0;i<3;i++) cout << vehicles[i].plateNumber << " ";
        cout << "\n";
    }
}

// â”€â”€ 12. Police Patrol
void showPolicePatrol() {
    cout << "\n"; displaySeparator('-',62);
    cout << " TRAFFIC POLICE PATROL SYSTEM [Hour: " << currentHour << ":00]\n";
    displaySeparator('-',62);
    cout << " " << left<<setw(4)<<"ID"<<setw(18)<<"Officer Name"
        <<setw(8)<<"Shift"<<setw(28)<<"Assigned Location"<<"Challans\n";
    displaySeparator('-',62);
    for (int i=0;i<MAX_OFFICERS;i++) {
        bool onDuty = (officerSchedule[i][currentHour]==1); // 2D array access
        cout << " " << left<<setw(4)<<officers[i].id
            <<setw(18)<<officers[i].name
            <<setw(8)<<officers[i].shift
            <<setw(28)<<interNames[officers[i].intersectionId]
            <<officers[i].challansIssued;
        cout << (onDuty ? " [ON DUTY]" : " [OFF DUTY]") << "\n";
    }
    // Count on-duty officers
    int onCount=0;
    for (int i=0;i<MAX_OFFICERS;i++)
        if (officerSchedule[i][currentHour]==1) onCount++;
    cout << "\n Officers Currently On Duty : " << onCount << " / " << MAX_OFFICERS << "\n";
}

// â”€â”€ 13. Construction Zones
void showConstructionZones() {
    cout << "\n"; displaySeparator('-',60);
    cout << " ROAD CONSTRUCTION ZONES & DETOURS\n"; displaySeparator('-',60);
    bool any=false;
    for (int i=0;i<MAX_CONSTRUCTION;i++) {
        if (!constructions[i].active) continue; any=true;
        cout << "\n Zone #" << constructions[i].id << ": " << constructions[i].location << "\n";
    }
}

```

```

    cout << " Lanes Reduced   : " << constructions[i].lanesReduced << "\n";
    cout << " Speed Limit Here : " << constructions[i].speedLimitZone << " km/h\n";
    cout << " Detour Route     : " << constructions[i].detourRoute << "\n";
    cout << " Est. Completion   : " << constructions[i].completionDays << " days\n";
    displaySeparator('.',60);
}
if (!any) cout << "\n No active construction zones.\n";
}

// â”€â”€ 14. Fuel Stations
void showFuelStations() {
    cout << "\n"; displaySeparator('-',62);
    cout << " FUEL STATION & EV CHARGING STATUS\n"; displaySeparator('-',62);
    for (int i=0;i<MAX_FUEL_STATIONS;i++) {
        cout << "\n [" << i+1 << "] " << fuelStations[i].name << "\n";
        cout << " Near      : " << fuelStations[i].nearIntersection << "\n";
        cout << " Petrol   : " << (fuelStations[i].petrolAvailable?"AVAILABLE":"OUT OF STOCK")
            << " @ PKR " << fixed<<setprecision(0)<<fuelStations[i].petrolPrice << "/L\n";
        cout << " Diesel   : " << (fuelStations[i].dieselAvailable?"AVAILABLE":"OUT OF STOCK")
            << " @ PKR " << fuelStations[i].dieselPrice << "/L\n";
        cout << " EV Charge : " << (fuelStations[i].evCharging?"YES":"NO");
        if (fuelStations[i].evCharging)
            cout << " Slots: " << fuelStations[i].evSlotsAvailable
                << "/" << fuelStations[i].evSlotsTotal << " free";
        cout << "\n";
        displaySeparator('.',62);
    }
}

// â”€â”€ 15. Bus Schedule
void showBusSchedule() {
    cout << "\n"; displaySeparator('-',65);
    cout << " PUBLIC TRANSPORT SCHEDULE & DELAYS\n"; displaySeparator('-',65);
    cout << " " << left<<setw(8)<<"Route"<<setw(14)<<"From"<<setw(14)<<"To"
        <<setw(10)<<"Scheduled"<<setw(10)<<"Delay"<<setw(12)<<"Passengers"<<"Status\n";
    displaySeparator('-',65);
    for (int i=0;i<MAX_BUS_ROUTES;i++) {
        cout << " " << left<<setw(8)<<busRoutes[i].routeName
            <<setw(14)<<busRoutes[i].from
            <<setw(14)<<busRoutes[i].to
            <<setw(10)<<busRoutes[i].scheduledHour
            <<setw(10)<<busRoutes[i].delayMinutes
            <<setw(12)<<busRoutes[i].passengerCount;
        if (busRoutes[i].cancelled) cout<<"CANCELLED\n";
        else if (busRoutes[i].delayMinutes>20) cout<<"DELAYED\n";
    }
}

```

```

        else if (busRoutes[i].delayMinutes>0) cout<<"SLIGHT DELAY\n";
        else cout<<"ON TIME\n";
    }
}

```

// â”€â”€ 16. Officer Roster

```

void showOfficerRoster() {
    cout << "\n"; displaySeparator('-',60);
    cout << " OFFICER DAY/NIGHT DUTY ROSTER\n"; displaySeparator('-',60);
    cout << " Day Shift (08:00 - 20:00): Officers 1-6\n";
    cout << " Night Shift(20:00 - 08:00): Officers 7-12\n\n";
    cout << " Officer      | Shift | Status at " << currentHour << ":00\n";
    displaySeparator('-',60);
    for (int i=0;i<MAX_OFFICERS;i++) {
        bool onDuty=(officerSchedule[i][currentHour]==1); // 2D array
        cout << " " << left<<setw(18)<<officers[i].name
            << "|" <<setw(6)<<officers[i].shift
            << "|" << (onDuty?"ON DUTY ":"OFF DUTY ") << "\n";
    }
    // Show next 6 hours using 2D array
    cout << "\n Schedule next 6 hours:\n";
    cout << " Officer      |";
    for (int h=0;h<6;h++) cout<<setw(4)<<(currentHour+h)%24<<"h";
    cout<<"\n";
    displaySeparator('-',60);
    for (int i=0;i<MAX_OFFICERS;i++) {
        cout << " " << left<<setw(18)<<officers[i].name<<"|";
        for (int h=0;h<6;h++)
            cout<<setw(4)<<(officerSchedule[i][(currentHour+h)%24]?"ON":"--"); // 2D array
        cout<<"\n";
    }
}

```

// â”€â”€ 17. Legal Checks

```

void showLegalChecks() {
    cout << "\n"; displaySeparator('-',62);
    cout << " LEGAL CHECKS: Drunk / License / Fitness / Seatbelt\n";
    displaySeparator('-',62);

    cout << "\n -- DRUNK DRIVING DETECTIONS (" << drunkCount << " found) --\n";
    for (int i=0;i<MAX_VEHICLES;i++) {
        if (!vehicles[i].drunkDriver) continue;
        cout << " [" << vehicles[i].plateNumber << " ] "

```

```

        << vehicles[i].type << " Lane:" << vehicles[i].lane
        << " Fine: PKR " << violationFine[4] << " | License suspended\n";
    }

```

```

cout << "\n -- WRONG-WAY DRIVING (" << wrongWayCount << " found) --\n";
for (int i=0;i<MAX_VEHICLES;i++) {
    if (!vehicles[i].wrongWay) continue;
    cout << " [" << vehicles[i].plateNumber << "] "
        << vehicles[i].type << " Lane:" << vehicles[i].lane
        << " Fine: PKR " << violationFine[3] << "\n";
}

```

```

cout << "\n -- NO SEATBELT / HELMET (" << noSeatbeltCount << " found) --\n";
for (int i=0;i<MAX_VEHICLES;i++) {
    if (!vehicles[i].noSeatbelt) continue;
    cout << " [" << vehicles[i].plateNumber << "] "
        << vehicles[i].type << " Fine: PKR " << violationFine[1] << "\n";
}

```

```

cout << "\n -- FITNESS CERTIFICATE EXPIRED (" << fitnessFailCount << " found) --\n";
for (int i=0;i<MAX_VEHICLES;i++) {
    if (!vehicles[i].fitnessExpired) continue;
    cout << " [" << vehicles[i].plateNumber << "] "
        << vehicles[i].type << " Fine: PKR 800\n";
}

```

```

cout << "\n -- LICENSE EXPIRED / SUSPENDED (" << licenseExpCount << " found) --\n";
for (int i=0;i<MAX_VEHICLES;i++) {
    if (!vehicles[i].license.expired && !vehicles[i].license.suspended) continue;
    cout << " [" << vehicles[i].plateNumber << "] "
        << vehicles[i].type << " License: " << vehicles[i].license.number
        << " Demerit: " << vehicles[i].license.demeritPoints
        << " Fine: PKR 2500\n";
}
}

```

// â”€â”€ 18. Final Report

```

void showFinalReport() {
    cout << "\n"; displaySeparator('= ',62);
    cout << " FINAL TRAFFIC MANAGEMENT REPORT\n";
    cout << " Hour: " << currentHour << ":00 | Weather: "
        << currentWeather.condition << "\n";
    displaySeparator('= ',62);
}

```

```

// 1. Fines
cout << "\n [1] FINES & CHALLANS\n"; displaySeparator('-',40);
cout << " Overspeed Challans : " << overspeedCount << "\n";
cout << " Drunk Driving : " << drunkCount << "\n";
cout << " Wrong-Way Driving : " << wrongWayCount << "\n";
cout << " No Seatbelt/Helmet : " << noSeatbeltCount << "\n";
cout << " Fitness Expired : " << fitnessFailCount << "\n";
cout << " License Expired : " << licenseExpCount << "\n";
cout << " TOTAL FINES COLLECTED: PKR " << totalFines << "\n";
cout << " TOLL REVENUE : PKR " << totalToll << "\n";

// 2. Worst intersection
cout << "\n [2] WORST CONGESTED INTERSECTION\n"; displaySeparator('-',40);
int worst=0;
for (int i=1;i<MAX_INTERSECTIONS;i++)
    if (intersections[i].congestionLevel>intersections[worst].congestionLevel) worst=i;
cout << " Location : " << intersections[worst].name << "\n";
cout << " Vehicles : " << intersections[worst].vehicleCount << "\n";
cout << " Congestion : "; displayBar(intersections[worst].congestionLevel,20); cout<<"\n";
cout << " Action : ";
if (intersections[worst].congestionLevel>0.85f) cout<<"Divert traffic. Deploy officers.\n";
else if (intersections[worst].congestionLevel>0.65f) cout<<"Extend green signal.\n";
else
    cout<<"No action needed.\n";

// 3. Overspeed by lane (1D array)
cout << "\n [3] VIOLATIONS BY LANE\n"; displaySeparator('-',40);
int lv[4]={0,0,0,0};
for (int i=0;i<MAX_VEHICLES;i++)
    if (vehicles[i].isChallan) lv[vehicles[i].lane-1]++;
for (int l=0;l<4;l++)
    cout << " Lane " << (l+1) << " (limit " << speedLimit[l] << " km/h): "
        << lv[l] << " violations\n";

// 4. Emergency summary
cout << "\n [4] EMERGENCY RESPONSE SUMMARY\n"; displaySeparator('-',40);
cout << " Emergency Units Deployed : " << emergencyCount << "\n";
for (int i=0;i<emergencyCount;i++)
    cout << " - " << emergencies[i].type
        << " at " << interNames[emergencies[i].intersectionId]
        << " | " << emergencies[i].responseTimeSec << "s\n";

// 5. Accidents
cout << "\n [5] ACCIDENT SUMMARY\n"; displaySeparator('-',40);

```

```

cout << " Total Accidents : " << accidentCount << "\n";
cout << " Weather Condition: " << currentWeather.condition << "\n";
cout << " Risk Level      : " << (currentWeather.accidentRisk?"HIGH":"NORMAL") << "\n";

```

```

// Overall city score
cout << "\n"; displaySeparator('= ',62);
cout << " OVERALL CITY TRAFFIC HEALTH SCORE\n";
float score=100.0f;
score -= accidentCount*10.0f;
score -= intersections[worst].congestionLevel*20.0f;
score -= overspeedCount*0.5f;
score -= drunkCount*3.0f;
score -= wrongWayCount*2.0f;
if (score<0) score=0;
cout << " Score : "; displayBar(score/100.0f,30); cout<<"\n";
cout << " Status: ";
if (score>=75) cout<<"GOOD  â€” City traffic well managed.\n";
else if (score>=50) cout<<"FAIR  â€” Some areas need attention.\n";
else cout<<"POOR  â€” Immediate intervention needed.\n";
displaySeparator('= ',62);
}

```

```

int main() {

```

```

    cout << "\n"; displaySeparator('= ',62);
    cout << " Initializing Smart Traffic Management System v3.0...\n";
    displaySeparator('= ',62);
    initializeAll();
    pauseScreen();
    int choice;
    bool running=true;
    while (running) {
        displayBanner();
        displayMenu();
        cin >> choice;
        switch(choice) {
            case 1: showWeatherStatus(); pauseScreen(); break;
            case 2: showAllVehicles(); pauseScreen(); break;
            case 3: showSpeedViolations(); pauseScreen(); break;
            case 4: showIntersectionStatus();pauseScreen();break;
            case 5: showSignalStatus(); pauseScreen(); break;
            case 6: showAccidentReport(); pauseScreen(); break;
            case 7: showTollSummary(); pauseScreen(); break;
            case 8: showEmergencyStatus(); pauseScreen(); break;
            case 9: showHourlyFlow(); pauseScreen(); break;

```

```
case 10: show3DSignalGrid();  pauseScreen(); break;
case 11: showVehicleSearch();  pauseScreen(); break;
case 12: showPolicePatrol();  pauseScreen(); break;
case 13: showConstructionZones();pauseScreen(); break;
case 14: showFuelStations();  pauseScreen(); break;
case 15: showBusSchedule();  pauseScreen(); break;
case 16: showOfficerRoster();  pauseScreen(); break;
case 17: showLegalChecks();  pauseScreen(); break;
case 18: showFinalReport();  pauseScreen(); break;
case 0:
    cout << "\n System shutting down. Goodbye!\n\n";
    running=false; break;
default:
    cout << "\n Invalid choice. Try 0-18.\n";
    pauseScreen();
}
}
return 0;
}
```